CIVE 580C8 Final Project Brief

Agentic AI and Leadership for Civil and Environmental Engineering Practice, Fall 2026.

Read this first

You will build one agentic AI system with your team between now and December 9, and you will present it to a panel of practicing engineers, utility leaders, and regulators. The capstone is 30 percent of your grade. It is the part of the course an employer will ask about.

This brief does three things. It gives you five defined project options, each built around a different technical problem, and a sixth option for a project of your own design that meets the same bar. It sets out what we expect from a team, week by week, and what the panel will grade. Team assignments are posted separately.

What you are being asked for is not a **demo**. A demo works once, on your machine, on the input you chose. You are building something a stranger can install from your runbook, an operator can use without you in the room, and a regulator can audit from your log. Those are three different problems, and the third one is the one that matters in our sector.

Ground rules

Stack

The local stack from the labs is the default: Ollama on a laptop with 16 GB of memory and no GPU. A team may use a cloud model for one component if it justifies the choice with measured numbers in the Lab 13 cost model (cost per run, latency, and energy).

Data

Use instructor-provided synthetic generators plus public open data (preferred). No client data and no data covered by a nondisclosure agreement, including data from your own employer. If you use a live external source, commit a cached local snapshot to your repository. The panel demo runs offline.

Numbers

The model never computes a number that ends up in a deliverable. Averages, volumes, deadlines, scores, and dollar figures are computed in plain Python you can point to. The model reads, retrieves, classifies, drafts, and flags. If you cannot show the line of code that produced a number, the number does not go in the report.

Every deliverable says what it does not do

Your agent, your adoption brief, and your panel presentation each name what the system does not do and where it breaks. A team that presents only what works has not finished the course.

One repository from day one

Code, prompts, evaluation cases, the scope page, the risk register, and the runbook all live in one repository. Commit history is how we assess individual contribution, so commit under your own name and commit often.

The project options

Five options are described below. Each one names a real user, the operational system it has to touch, and the one problem that will make it hard. They are deliberately different from each other: one extracts rules from a permit, one assembles an incident report from several sources, one classifies records at scale, one is triggered by an outside event, and one wraps a deterministic engineering calculation. Read all five before you choose. The one that sounds least glamorous may be the one that teaches you the most.

Every option names a system the agent has to connect to: a laboratory information management system (LIMS), a geographic information system (GIS), a billing system, a historian, or a field inspection app. That is not decoration. An agent that touches none of these is a chatbot with an engineering vocabulary, and the panel will say so.

The five defined options are all water and wastewater problems, because that is where the labs, the synthetic data, and the guest speakers sit. **Option 6 lets a team bring a problem from another part of civil and environmental practice, as long as it meets the criteria listed there.**

Option 1. Permit monitoring plan and chain-of-custody agent

The problem. A discharge permit describes monitoring in prose and tables: which parameters, how often, by what analytical method, at which outfall. Someone has to turn that into a sampling calendar, keep track of holding times between the bottle and the laboratory, and prove afterward that every required sample was taken. When a sample is missed or held too long, the result is a permit violation for a sample that was never analyzed.

Your user. The compliance coordinator or the field sampling lead at a treatment plant. They plan next month in the last week of this month, and they want to know two things: what is due, and what went wrong last month.

Data. A synthetic discharge permit with a monitoring table, a synthetic LIMS sample log with realistic gaps, and the analytical method and holding time requirements in 40 CFR Part 136 (a real, public, citable source).

What your architecture has to do:

- Retrieval over the permit that turns the monitoring table into structured rules, with the permit clause cited on every rule (Lab 6). A rule without a citation does not exist.
- A deterministic scheduler that generates the sampling calendar and the holding time clock from those rules. The model proposes rules; Python computes dates.
- An output contract for the monthly completeness check: for every required sample, taken on time, held within limit, or ESCALATE with the reason (Lab 5).
- An audit log recording which permit version and which rules produced each calendar (Lab 10).

Where you will get stuck. Permit language. "Quarterly" starting when? Does "weekly" mean any 7 day window or a calendar week? A holding time that expires on a Sunday? Your model will happily guess. Your job is to make it say "cannot determine" and route the clause to a person.

Questions you must answer. What does the LIMS export look like, and who runs it? Where does the calendar go, into a calendar system, a work order, or a printout on the wall? Who is accountable when a sample is missed, and how does your log help them?

What it does not do. It does not interpret a result and it does not decide whether a result is a violation. It never writes to the LIMS.

How you will know it works. Missed and late samples detected against a seeded set, and the number of permit clauses the agent correctly refused to interpret.

Panel-ready looks like: one permit, one month of messy sample data, a calendar the coordinator would use, and one seeded holding time failure the agent catches and one ambiguous clause it escalates.

Option 2. Sanitary sewer overflow report agent

The problem. When a sewer overflows, the utility must report it to the state on a fixed clock: under 40 CFR 122.41(l)(6), noncompliance that may endanger health or the environment is reported orally within 24 hours and in writing within 5 days. The report needs a volume estimate, a cause, the receiving water, the rainfall at the time, and a narrative. The person writing it is the same person who was out at the manhole at two in the morning.

Your user. The collection system supervisor, the morning after. They have a field log, a few photos, and a form that asks for numbers they have to estimate.

Data. A synthetic overflow incident log and field notes, a synthetic collection system GIS (manholes, pipes, outfalls, receiving waters), public rainfall and stream gauge data (National Weather Service, USGS NWIS), and your state's overflow report form as the output shape.

What your architecture has to do:

- Multi-tool orchestration (Lab 7): an incident log query, a rainfall lookup, a GIS lookup for the nearest receiving water and its distance, and a form filler. The router's tool choice log is a graded artifact.
- A deterministic volume estimator that applies a published method (for example, a manhole overflow chart or a duration and flow rate calculation) to the operator's inputs and shows its work. The model never produces the volume.
- An output contract matching the state form field by field, with ESCALATE on any missing input that changes the report category (Lab 5).
- Deadline tracking computed from the discovery time, and an audit log of every draft and who approved it (Lab 10).

Where you will get stuck. The volume. Every input to the estimate is uncertain and the number decides how serious the event is. Your agent must present the estimate with its method and its range, never as a single confident figure, and it must refuse to estimate when the inputs are missing.

Questions you must answer. Which form, in which state? Where do the operator's notes come from, a paper log, a phone app, a work order system? Who signs the report, and what do they need to see to sign it?

What it does not do. It does not send anything to the regulator and it does not write public notifications. It drafts for a person who signs.

How you will know it works. Field completeness and volume estimate agreement against seeded incidents with known answers, and hours from discovery to a draft ready for signature.

Panel-ready looks like: one replayed overflow, from field notes to a signed-ready draft, with the deadline clock visible and one seeded case where the agent refuses to estimate volume.

Option 3. Lead service line inventory agent

The problem. The Lead and Copper Rule Improvements (LCRI, finalized by the U.S. Environmental Protection Agency in October 2024) require water systems to hold a baseline service line inventory by November 1, 2027, and to replace lead lines within 10 years. The material of most older lines is recorded, if at all, in tap cards, scanned permits, plumber notes, and decades of free-text work orders. Nobody has time to read them all, and "unknown" is the category that costs the most.

Your user. The water system's inventory lead and the GIS analyst. They need to shrink the unknown bucket and defend every reclassification to the state.

Data. A synthetic corpus of service records with realistic messiness (abbreviations, conflicting entries, missing years), a synthetic parcel and service line GIS layer, and the public EPA inventory guidance for the required categories.

What your architecture has to do:

- Bulk classification of each record into the required categories (lead, galvanized requiring replacement, non-lead, unknown) with a confidence score and the evidence quoted from the record.
- A human review queue for every classification below threshold, and a rule that conflicting evidence means unknown, never a coin flip (Lab 10).
- Aggregation in DuckDB or SQL after classification. The model classifies; SQL counts.
- A GIS join that maps the unknowns and a prioritization score computed by an explicit formula the user can read and change.

Where you will get stuck. Proving the classifier is right at scale. You cannot hand-check every record, so you need a sampling-based validation with a stated confidence interval (Lab 8 applied honestly). That is more statistics than you expect, and it is the part the state will ask about.

Questions you must answer. What does the record export look like, and how do scanned documents get into it? Does a reclassification write back to the inventory system or live in a parallel table? Who works the review queue, and is that job funded?

What it does not do. It does not verify a material. A record that says copper is evidence, not proof, and the agent says so. It does not decide the replacement order.

How you will know it works. Classification accuracy on a hand-labeled sample with a confidence interval, the size of the unknown bucket before and after, and how many reclassifications survive a reviewer.

Panel-ready looks like: the classified inventory, the map of unknowns, the validation statistics, and one record the agent correctly refused to classify.

Option 4. Construction stormwater inspection agent

The problem. Under the EPA 2022 Construction General Permit, a site must be inspected on a fixed schedule: either every 7 calendar days, or every 14 calendar days and within 24 hours of a storm of 0.25 inch or more. Each inspection produces a report, each deficiency has a corrective action deadline, and each missed inspection is a permit violation. Across a dozen active sites the inspector spends as much time on paperwork and rain tracking as on inspection.

Your user. The site stormwater inspector and the contractor's compliance manager. They work from a truck with a tablet.

Data. A synthetic site register with locations and control measures, a synthetic inspection and deficiency log, and public rainfall observations (National Weather Service or a state mesonet) cached to the repository.

What your architecture has to do:

- An event-triggered loop: a deterministic rain check against each site's location decides whether a storm inspection is due, and by when. The model never decides whether it rained.
- Retrieval over the site's stormwater pollution prevention plan (SWPPP) to draft the inspection checklist for that site's control measures, with the plan section cited (Lab 6).
- An output contract for the inspection report, with corrective actions carrying a computed deadline from the permit and ESCALATE when a deficiency could be a discharge (Lab 5).
- A governance policy (Lab 12) that forbids the agent from marking an inspection complete. Only the inspector does that, and the test proving it is a graded artifact.

Where you will get stuck. The gap between a weather station and a site. Rain at the airport is not rain at the site. Your agent has to state which observation it used, how far away it is, and what it does when the data is missing or late.

Questions you must answer. Which inspection app or form does the contractor already use, and how does your report get into it? Who is the certified inspector of record, and how does your tool keep them in that role? What happens on a Sunday storm?

What it does not do. It does not inspect. It does not certify. It prepares the inspector, drafts the report, and keeps the clock.

How you will know it works. Missed storm inspections detected against seeded rainfall, corrective action deadlines computed correctly, and inspector minutes per report before and after.

Panel-ready looks like: a replayed rainy week across several sites, the agent's due list, a drafted report for one site, and the policy test showing the agent cannot close an inspection.

Real and apparent losses calculated with python.
Grades of confidence of inputs based on model judgement?

Option 5. Non-revenue water audit agent

The problem. A water utility produces one volume of water and bills another, and the gap is leakage, meter error, theft, and data handling error in unknown proportions. The AWWA M36 water audit method splits that gap into real and apparent losses and grades the confidence in every input. Most small utilities do the audit once a year, by hand, from a billing export and a production meter log, and the result depends heavily on assumptions nobody writes down.

Your user. The utility's finance or operations manager who fills out the annual audit, and the engineer who has to explain the result to a board.

Data. A synthetic billing export by customer class with realistic lag and estimated reads, a synthetic production meter record with a calibration history, and the public AWWA free water audit method as the calculation and grading framework.

What your architecture has to do:

- The full water balance computed deterministically in Python from the M36 definitions. The model never computes a volume or a loss.
- A structured interview: the agent asks the user the data validity questions, records the answers with their justification, and computes the validity grade from them (Lab 5).
- Anomaly flags on the inputs (a billing month that is too low, a meter that has not been tested in years) with the evidence and a suggested question, never a corrected number.
- A sensitivity table showing which assumption moves the loss figure most, and an audit log of every input and grade (Lab 10 and Lab 13).

Where you will get stuck. The audit is only as good as its worst input, and the agent will be tempted to fill gaps with typical values. It must not. Every default the user accepts is recorded as a default, and the sensitivity table shows what it costs.

Questions you must answer. What does the billing export look like, and how are estimated reads marked? Who owns the production meter, and when was it last tested? What decision does the audit feed, a rate case, a leak detection budget, a state filing?

What it does not do. It does not find leaks and it does not set rates. It produces a defensible audit and a list of the questions that would make next year's audit better.

How you will know it works. Water balance results that match a hand-worked reference audit, validity grades a reviewer agrees with, and the number of unstated assumptions the agent surfaces.

Panel-ready looks like: a complete audit for one synthetic utility, the sensitivity table, and one seeded bad input the agent flags rather than absorbs.

Option 6. Create your own

A team may propose a project of its own, in water or in any other part of civil and environmental practice: transportation, structures, geotechnical, construction, environmental permitting, solid waste, air, or site remediation. The bar is the same as for the five defined options, and it is set out below so that you can check your idea against it before you write the executive summary.

A project of your own is approved only if it meets every one of the following criteria:

1. A named user in a named role, and a task that person does today, repeatedly, by hand. Not "engineers" and not "the public." One role, one recurring task, and you can describe what their week looks like now.
2. A multi-step workflow. The agent must plan and execute at least three distinct steps using at least two tools (for example, a database query, a document retrieval, a calculation, a form filler, a GIS lookup). A single prompt that summarizes a document is not an agent.
3. A real operational system it has to connect to, named specifically: a LIMS, a CMMS, a GIS, a historian, a billing system, a project management or document control system, a field inspection app, or a regulator's reporting portal. You must be able to describe what its export looks like and who runs it.
4. A governing document to retrieve over and cite: a permit, a standard, a specification, a regulation, a design manual, or a contract. Every finding your agent produces traces to a clause in it.
5. Numbers computed in code. Any quantity, date, score, or dollar figure in the output is produced by deterministic code you can point to. The model reads, retrieves, classifies, drafts, and flags.
6. A stated escalation. There is at least one condition under which the correct output is "cannot determine" or "needs a person," and you can name it before you build.
7. A hard boundary. There is something the agent is never allowed to do (write to the system of record, sign, submit, close, or control), and it is enforced in code and proven by a test, not by a sentence in the prompt.
8. Data you are allowed to use. Synthetic data you generate, plus public data with a cached snapshot in the repository. No client data, no employer data, no data under a nondisclosure agreement. If you cannot get realistic data for the problem, you cannot do the project.
9. A measurable result. One or two numbers that say whether it works, measured against seeded cases with known answers, and one number the adoption brief can carry (hours, cycles, errors caught, or dollars).
10. Sized for the calendar. One complete path from input to output demonstrable by November 4, and everything else finished by the November 25 freeze. If the honest answer is that the first working path arrives in December, the scope is too large.

Two things disqualify a proposal on their own: a project that needs data you do not have permission to use, and a project whose agent has a write path into a live system. A proposal that needs a cloud model to work at all is not disqualified, but it must say so in the executive summary and justify it with the Lab 13 numbers later.

How to pick

Each team submits an executive summary by **Friday, October 2**, through Canvas. One page, no slides, written for a reader who has not seen this brief.

The executive summary contains 5 things:

- The problem, in one paragraph, in the user's words.
- The user, named as a role, and what their week looks like today.
- What the agent will do, in three to five steps, and which tools and system it touches.
- The one thing most likely to make this project fail, and what you will do about it.
- The data sources, and where each one actually comes from.

From October 2nd, every lab artifact you build (the multi-tool chain in Lab 7, the evaluation suite in Lab 8, the adversarial tests in Lab 9, the approval gates and audit log in Lab 10) is built against your own project's data where the lab allows it.

What we expect from a team

Roles

Every team covers four roles, and every role has one named owner in the repository README. With three people, someone owns two roles. The roles are a hedge against your own strengths: the failure we see most often is a team that all likes writing code, producing a beautiful build with no evaluation suite and no adoption case.

- Builder: code, tools, orchestration, and the runbook.
- Domain lead: whether the answer is actually right under the permit, the rule, or the method.
- Verifier: the evaluation suite, the adversarial cases, and the failure register.
- Adoption lead: the two-page brief, the cost model, and the panel pitch.

Assign roles by what each person is good at, not by seniority, and rotate who presents.

Online section

The online team works to the same deliverables and the same dates. Meet weekly on your own schedule. Your progress demo on December 2 and your panel presentation are delivered live over video with screen sharing, and the recorded demo is due before the panel.

The executive summary, dates, and deliverables are the same, and the scope is smaller: one user, one tool chain, an evaluation suite of at least 15 cases with at least 3 adversarial cases, and a one-page adoption brief in place of the two-page brief.

The cold start test

On December 4, another team runs your repository from your runbook in 15 minutes with no spoken help from you. Every question they have to ask is a documentation defect, and the count is part of your documentation grade.

The final panel (subject to change based on panelist schedules)

Format: 25 minutes per team, in the order announced on December 2. A 12 minute presentation, an 8 minute live demo, and 5 minutes of panel questions. The panel is drawn from a utility operations leader, a consulting principal, a regulator or compliance officer, and a data or AI practitioner from outside the water sector. Each asks a different kind of question, and they will find the soft spot.

The presentation has seven parts, and both halves of the course carry equal weight:

11. The problem, in your user's words.
12. What the system does, shown live.
13. How it is grounded, and how it fails.
14. Evaluation results, including what it still gets wrong.
15. Governance: trust boundaries enforced in code, autonomy level, audit trail.
16. The business case: cost, total cost of ownership, and measured energy.
17. The adoption path: who champions it, who resists, and what the first 90 days look like.

Add one slide on what you would do differently. The teams that answer that well are the ones the panel wants to hire.

What is graded

Table 2. Capstone rubric (100 points, 30 percent of the course grade)

Criterion	Points	What earns full marks
System works, live, under demo conditions	20	Runs end to end on real-shaped data, handles one input correctly and one badly on purpose.
Grounding and evaluation rigor	20	Every number traces to code, every claim to a source. Pass rates by category, failures named.

Criterion	Points	What earns full marks
Governance in code	15	Trust boundaries enforced by tests, audit log with working replay, policy file that blocks a violation.
Integration and deployment realism	10	Names the actual system, interface, and export cadence. "It connects to the LIMS" scores zero.
Business case	15	Cost per run, total cost of ownership, measured energy, and a sensitivity band.
Adoption brief and change management	10	Two pages a decision maker could act on, naming what the tool does not do.
Presentation and answering the panel directly	10	Answers the question asked. Says "we do not know" when that is the answer.

Assessed separately: documentation, from the cold start test, and individual contribution, from commit history and a short peer assessment. A team of three that is really a team of one will not be able to hide it.

Questions you should be able to answer at any point

- What happens when the export is a day late, or the rain data is missing?
- Walk me through the exact screen your user sees, on the day they need it.
- What is this agent not allowed to do, and where is that enforced?
- How much energy does one run cost, and how do you know?
- If I gave you this tool at my utility tomorrow, what would break first?

Scope narrow. Verify hard. Design for the person who has to trust it.

References

Regulations and guidance

U.S. Environmental Protection Agency. 40 CFR Part 122.41(l)(6), Twenty-four hour reporting.

U.S. Environmental Protection Agency. 40 CFR Part 136, Guidelines Establishing Test Procedures for the Analysis of Pollutants, Table II, Required Containers, Preservation Techniques, and Holding Times.

U.S. Environmental Protection Agency. 2022 Construction General Permit, Part 4, Inspections, and Part 5, Corrective Actions.

U.S. Environmental Protection Agency. Lead and Copper Rule Improvements, final rule, October 2024.

Methods

American Water Works Association. Manual M36, Water Audits and Loss Control Programs, and the AWWA Free Water Audit Software.